

AI 시스템반도체 SW 개발자 2기

Smart_Fan
Linux_Driver

고지현 박서희 이규현 이호윤

목 차

01 프로젝트 개요

02 핵심 기술 개념

03 시스템 설계

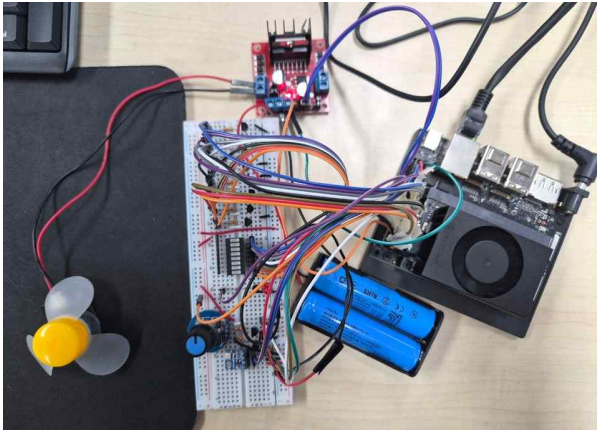
04 디바이스 드라이버 구현
DC 모터 · EC11 · LED 바 · BMP180

05 통합 동작 및 검증

06 결과 및 발전 방향

프로젝트 소개

Jetson Orin Nano 기반 온도 연동 스마트 선풍기



온도에 따라 스스로 바람을 조절

BMP180 센서 모듈로 실내 온도를 측정해 DC 모터의 풍량을 자동으로 조절하고, EC11 로터리 엔코더로 수동 조작과 모드 전환이 가능한 스마트 선풍기

센서, 모터, 엔코더, LED 바를 모두 직접 작성한 리눅스 커널 드라이버로 제어하고, 사용자 공간 앱이 이를 하나로 묶어 동작

#리눅스커널드라이버 #JetsonOrinNano #AUTO·MANUAL

하드웨어 구성

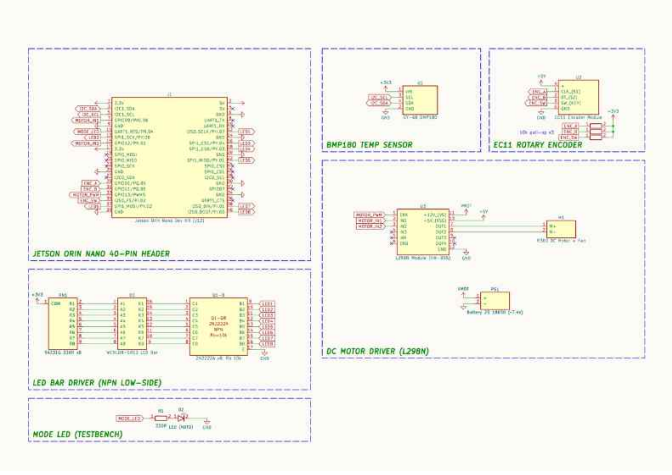
사용 부품과 Jetson 40핀 헤더 연결

부품	역할	연결 (헤더 핀)
BMP180 (GY-68) 대기압 센서 모듈	실내 온도 측정	I2C 버스 7 · 핀 3(SDA) / 5(SCL)
R300 DC 모터 + L298N 드라이버	풍량 (PWM 1 kHz)	핀 33(PWM) · 7(IN1)/ 15(IN2)
EC11 로터리 엔코더	모드 전환 · 풍량 조절	핀 29(S1) / 31(S2) / 35(SW)
8칸 LED 바 + 330Ω×8 저항 어레이	온도 · 풍량 표시	핀 12·13·16·18·22·37·38·40
Jetson Orin Nano Developer Kit	메인 보드 (JetPack 6)	L4T R36.5 · Linux 5.15

※ JetPack 6에서는 헤더 핀이 출력 꺼짐(tristate) 상태로 부팅되므로, 사용하는 핀을 jetson-io에서 GPIO / PWM으로 설정

회로도 및 설계 이유

전체 배선과 부품·연결 방식을 선택한 근거



왜 이렇게 설계했나

- 3.3V 로직 통일 : 젯슨 GPIO가 3.3V이므로 엔코더 전원은 5V, 신호선은 3.3V로 풀업하여 구성.
- 모터 전원 분리 : L298N에 모터 구동을 위한 별도 전원 공급, GND만 공통으로 묶음
- 330Ω 저항 어레이 : LED 8칸 전류를 칸당 약 4 mA로 제한
- 하드웨어 PWM(핀 33) : CPU 부담 없이 1 kHz 파형 생성
- 엔코더 KEY Active-Low : 풀업 상태에서 누르면 Low

Mutex

공유 자원에 대한 동시 접근을 막는 상호 배제 기법

동시에 들어오는 요청

요청 A PWM 70% 설정	→	사용 권한 획득 → GPIO·PWM 설정
요청 B 모터 정지	→	권한 획득 대기 → A 완료 후 정지



왜 필요한가

작업 순서가 섞이는 경쟁 상태 방지

모터 방향·PWM 출력을 한 작업으로 보호

Mutex 잠금으로 보호 구간을 만들고 반드시 해제

기본 동작

mutex_init()
초기화
→
mutex_lock()
잠금 획득
→
공유 자원 작업
임계 구역
→
mutex_unlock()
잠금 해제

프로젝트 적용

DC 모터

ioctl · close 요청
motor->lock으로 GPIO·PWM 변경 보호

BMP180

IIO read_raw 요청
data->lock으로 측정·보정 순서 보호

대기 중 잠들 수 있는 잠금 · 인터럽트/타이머 문맥에서는 사용 불가

디바이스 트리 (DT)

하드웨어 정보를 커널 코드에서 분리

기존: 보드별 정보를 커널 코드에 작성

보드 A 핀·주소 → 보드 A용 커널 코드

보드 B 핀·주소 → 보드 B용 커널 코드

DT: 보드별 정보를 데이터로 전달

A.dtb 보드 A 정보 ↘

B.dtb 보드 B 정보 ↗

공통 커널 이미지 + 장치 드라이버

왜 사용하는가

보드가 늘어도 커널 코드 중복을 줄임

핀·주소 변경은 DTB 설정으로 관리

같은 장치인 경우 여러 보드에서 하나의 드라이버 재사용 가능

디바이스 트리 파일 구성

텍스트로 작성하고 바이너리로 전달

DTSI .dtsi		DTS .dts		DTB .dtb
SoC와 여러 보드가 공유하는 설정	+	보드의 장치·배선·주소·인터럽트 기술	→	dtc가 만든 바이너리 부팅 때 커널에 전달

프로젝트에서 사용한 오버레이

기존 DTB에 장치 설정을 덧붙이는 파일: .dtbo

.dts 작성 센서·GPIO·PWM → dtc 컴파일 .dtbo 생성 → 부팅 시 오버레이 기본 DT에 적용 → compatible 드라이버 탐색 → probe() 초기화 시도

예: BMP180 노드의 compatible = "custom,bmp180" · I²C 주소 reg = <0x77>

ioctl

장치별 제어 명령을 전달

ioctl의 역할



장치를 지정하고 요청 전달

드라이버가 해당 동작 수행

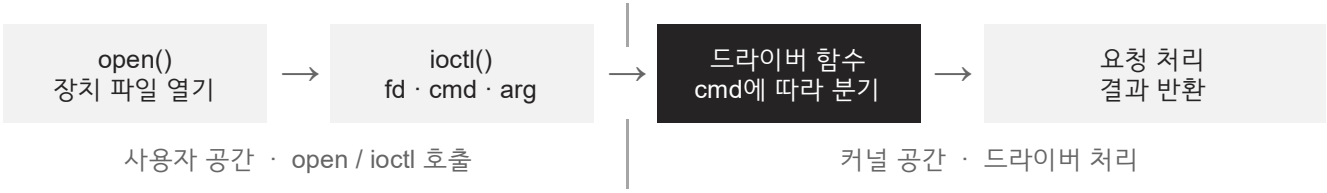
왜 사용하는가

설정값 전달·상태 조회처럼 read()/write()로 표현하기 어려운 작업을 요청
사용자 프로그램은 하드웨어를 직접 제어하지 않고, 드라이버가 정의한 명령을 통해 장치를 제어

ioctl 동작 원리

명령과 인자 전달

호출과 처리 순서



명령 코드 : 전달 방향에 따라 구분, 사용자 프로그램 기준

_IO _IOW _IOR _IOWR

_IO: 인자 없음 _IOW: 사용자 → 커널
_IOR: 커널 → 사용자 _IOWR: 양방향

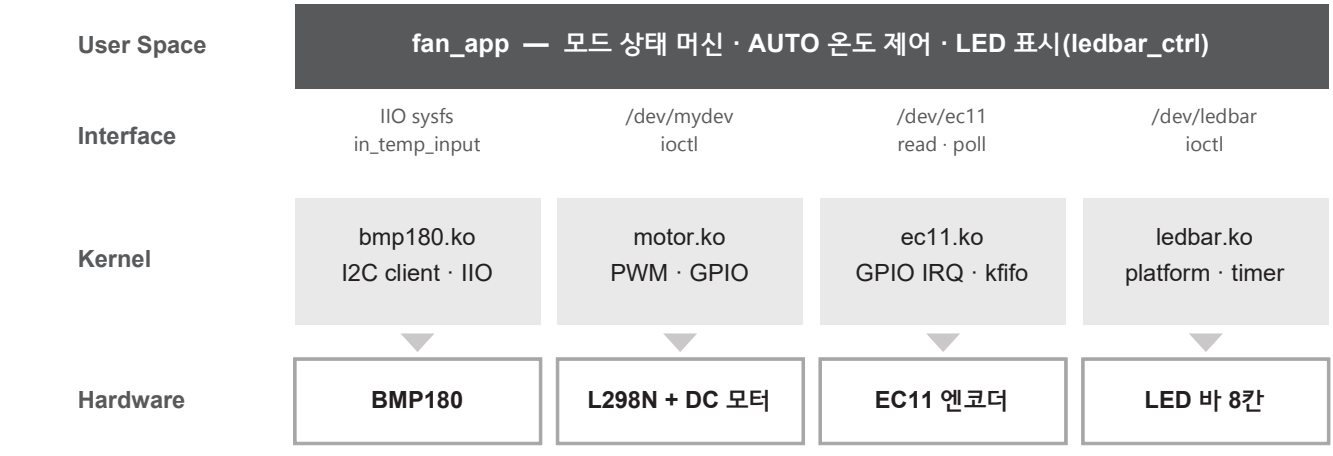
호출 인자

fd: 열린 장치 cmd: 동작 종류

arg: 전달 데이터 반환값: 처리 결과

시스템 아키텍처

사용자 공간 앱 ↔ 커널 드라이버 ↔ 하드웨어



유저 공간 vs 커널 공간

제어 정책은 앱이 결정하고, 드라이버는 하드웨어를 안전하게 제어

User Space · fan_app

- 기능 요구에 따라 제어 정책 결정
- 프로세스마다 독립된 가상 메모리 사용
- 오류가 나도 다른 앱 · 커널로 확산 억제
- 시스템 콜로 장치 기능을 안전하게 요청
- 앱만 교체해 동작 · UI를 유연하게 변경

시스템 콜 경계

◀ read · poll
입력 이벤트

ioctl ▶
duty · LED 칸 수

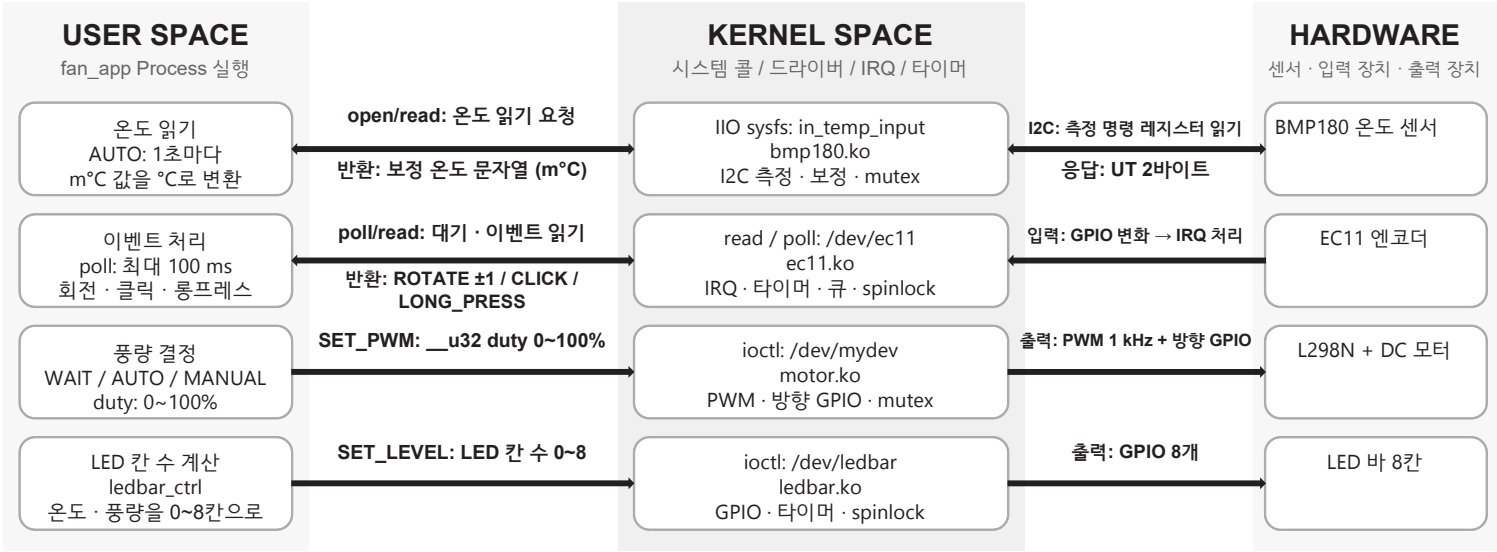
◀ sysfs (IIO)
온도 값

Kernel Space · 드라이버

- 하드웨어 · 메모리에 높은 권한으로 접근
- 드라이버가 장치 제어 절차를 일관되게 제공
- 여러 프로세스의 요청과 공유 자원 관리
- 오류 발생 시 시스템 전체에 영향 가능
- 최소 기능만 두어 안정성과 보안 확보

정책(Policy)은 유저 공간, 메커니즘(Mechanism)은 커널 공간
정책과 매커니즘의 분리→ 드라이버 수정 없이 앱만 바꿔 동작 변경

USER/KERNEL SPACE에서 데이터의 흐름



디렉토리 구조

드라이버별로 디렉토리를 나누고 Application에서 통합

```
linux_driver_project/
├── Application/
│   fan_app.c · fan_app.h · Makefile
└── drivers/
    ├── bmp180/     bmp180.c · bmp180.h · overlay
    ├── dc_motor/   motor.c · motor_ioctl.h
    ├── ec11/       ec11.c · ec11_ioctl.h
    └── ledbar/     ledbar.c · ledbar_ioctl.h
                    ledbar_ctrl.c · fan_globals.h
```

구성 원칙

- 드라이버마다 독립된 Makefile(obj-m)로 .ko 빌드
- 앱과 커널이 공유하는 ioctl 헤더는 각 드라이버 폴더에 두고 앱에서 include
- 팀 공용 전역변수(TEMP, PWM_DUTY, FAN_LEVEL, fan_mode)는 fan_globals.h로 통일
- 드라이버 단독 테스트 앱(apptest, ec11_test 등) 함께 제공



디바이스 트리 (DT)

하드웨어 정보를 코드에서 분리

① .dts 작성 핀·버스 정보 기술	② dtc 컴파일 → .dtbo	③ /boot 복사 extlinux OVERLAYS	④ 부팅 시 기본 DT에 병합	**⑤ compatible 매칭 → probe() 호출**

드라이버	compatible	DT에 기술한 하드웨어
bmp180	custom,bmp180	i2c@c250000 하위 · reg = <0x77>
dc_motor	smartfan,dc-motor	in1 / in2-gpios · pwms = <&pwm5 0 1000000>
ec11	smartfan,ec11	a / b / key-gpios (active-low)
ledbar	team,ledbar	led-gpios × 8 · pinctrl (tristate 해제)

드라이버 코드에는 compatible만 두고 핀 번호는 DT에 → 배선을 바꿔도 드라이버 재컴파일 불필요

동작 시나리오

엔코더 버튼 하나로 3가지 모드 전환

WAIT | **AUTO** | **MANUAL**

짧게 클릭 : WAIT → 직전 모드로 복귀, AUTO ↔ MANUAL 전환 | 2초 길게 누르기 : 어느 모드에서든 → WAIT

- 모터 정지 (duty 0%)
- LED 바 전부 꺼짐
- 직전 모드를 기억했다가 클릭 시 복귀

- 1초마다 BMP180 온도 측정
- 22°C 이상 ON / 20°C 미만 OFF
- 22 → 32°C 구간 duty 40 → 100%
- LED 바 = 온도 (21°C 1칸 ~ 28°C 8칸)

- 노브 1칸 = duty ±5%
- duty 0 ~ 100% 범위 제한
- LED 바 = 풍량 (duty × 8 / 100, 올림)

DC 모터 드라이버

Device Tree + L298N GPIO/PWM

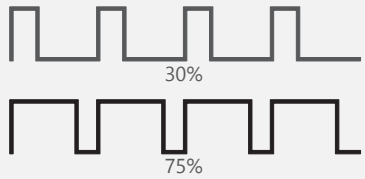
핵심 구현

- DT compatible로 platform_driver 매칭
- probe()에서 GPIO·PWM 리소스 획득
- miscdevice /dev/mydev (동적 minor)
- PWM 1 ms (1 kHz), duty 0~100% 적용
- close()·remove()에서 모터 정지

앱 인터페이스

```
__u32 duty = (__u32)percent;
ioctl(fd, MY_IOCTL_CMD_SET_PWM, &duty);
```

PWM HIGH 비율



모터 드라이버 구조

DT platform driver + miscdevice

fan_app · AUTO / MANUAL 듀티 결정

↓ open · ioctl(SET_PWM) · close

/dev/mydev · miscdevice + file_operations

↓ private_data → motor_dev

motor.ko · platform_driver + ioctl 처리

↓ devm_gpiod_get() · devm_pwm_get()

DT GPIO·PWM → L298N IN1·IN2·ENA → 팬

probe() 초기화

① DT compatible 매칭

② motor_dev 할당 · mutex 초기화

③ IN1·IN2 GPIO / motor PWM 획득

④ PWM OFF · misc_register()

miscdevice: 동적 minor로 /dev/mydev

GPIO·PWM: devm_*으로 자원 관리

사용자 공간 명령 처리

fan_app → miscdevice → ioctl

듀티 명령 전달

- ① fan_app이 AUTO/MANUAL 듀티 결정
- ② /dev/mydev에 SET_PWM ioctl 호출
- ③ copy_from_user()로 __u32 값 복사
- ④ motor->lock으로 출력 변경 보호
- ⑤ motor_set_speed(motor, duty) 호출

PWM 듀티 계산

주기 1 ms = 1 kHz

HIGH 시간 = 주기 × duty / 100

듀티 적용 예

30% → 0.3 ms HIGH

75% → 0.75 ms HIGH

동기화와 안전 정지

motor->lock · ioctl / release / remove

동시 요청 보호와 정지 경로

- probe()에서 장치별 mutex 초기화
- ioctl(): 듀티 설정 중 GPIO·PWM 보호
- release(): 장치 닫을 때 정지 처리 보호
- remove(): 등록 해제 후 잠금·정지
- PWM 적용 실패 시 motor_stop() 호출

mutex로 상태 변경 직렬화

```
mutex_lock(&motor->lock);
motor_set_speed(motor, duty_percent);
mutex_unlock(&motor->lock);
```

잠금을 공유하는 경로

ioctl() → 듀티 설정 보호

release()·remove() → 정지 처리 보호

EC11 로터리 엔코더

GPIO 인터럽트 + 커널 타이머 + 이벤트 큐

핵심 구현

- 선풍기의 조작부로 이용
- 3개 핀(S1 · S2 · KEY)의 변화를 인터럽트로 감지
- 노브의 회전은 디텐트(멈추는 위치)에서만 확정해 떨림을 제거
- KEY(버튼) : 20ms 디바운스 타이머 + 2s 롱프레스 타이머
- 이벤트는 kfifo에 쌓고, 앱은 poll() · read()로 수신

이벤트 → 앱 동작

ROTATE (±1)
MANUAL에서 duty ±5%

CLICK
AUTO ↔ MANUAL, WAIT 해제

LONG_PRESS (2 s)
WAIT으로 전환 (모터 정지)

EC11 드라이버 구조

platform driver + 문자 디바이스



probe() 순서

① GPIO 3개 획득 · IRQ 번호 변환

② 현재 A/B 상태 = 기준 디텐트

③ cdev 등록 → /dev/ec11 생성

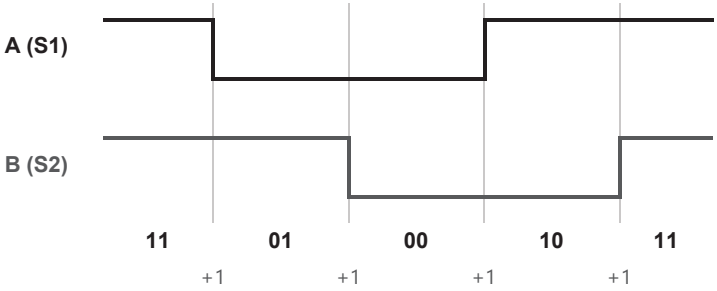
④ IRQ 3개 등록 (맨 마지막)

실패 시 goto로 역순 정리

GPIO · IRQ는 devm_*으로 자동 반납

회전 방향 판정

쿼드러처 디코딩 + 디텐트 확정



디텐트(11) 도착 → acc = +4 → CW 1칸 확정

접점 떨림 11 → 01 → 11 → 01 …
+1 −1 +1 −1 → acc ≈ 0 → 무시

판정 규칙

① 상태가 바뀔 때마다 ±1 누적
acc += quad_table[이전 · 현재]
② 디텐트(11)에 도착하면
acc ≥ +2 → CW
acc ≤ −2 → CCW
그 외 → 떨림으로 무시
③ acc = 0 초기화

인터럽트는 S1 · S2 양쪽 엣지에서 발생
판단은 모두 ISR 안에서 처리

버튼 처리와 이벤트 전달

커널 타이머 + kfifo + wait queue

KEY 입력 처리

엣지 IRQ
누를 때 · 뗄 때 모두 발생

→

디바운스 20 ms
엣지마다 타이머 재시작
안정되면 한 번만 읽기

→

눌림 확정
롱프레스 타이머 시작

→

결과 판정
2 s 전에 떼면 → CLICK
2 s 유지 → LONG_PRESS

이벤트 전달 (커널 → 사용자)

ISR · 타이머
이벤트 구조체 생성
ROTATE · CLICK · LONG

→

kfifo (64칸)
인터럽트 중에도
입력을 잃지 않고 저장

→

wait queue
자고 있던 read() 깨우기
CPU 점유 없이 대기

→

read() · poll()
fan_app이 최대 16개씩
한 번에 수신

ISR · 타이머 · read()가 같은 데이터를 공유 → spinlock_irqsave로 동시 접근 보호

LED 바 드라이버

platform driver + misc device + 커널 타이머

핵심 구현

- 선풍기의 상태를 시각적으로 표현하는 역할
- GPIO 8개를 배열로 받아(gpiod_get_array) 한 번에 갱신
- /dev/ledbar : write(텍스트) · read(상태) · ioctl(SET_LEVEL · SET_ANIM)
- timer_list 애니메이션 : WAVE(전원 ON) · BLINK · PROGRESS
- sysfs level · pattern · anim 으로 셸에서 바로 테스트
- 모듈 해제 · 시스템 종료 시 자동 소등

표시 규칙



예) AUTO · 25.3°C → 5칸

AUTO : 22°C 1칸 … 32°C 이상 8칸
(경계 ±0.2°C 히스테리시스)
MANUAL : duty × 8 / 100 (올림)
WAIT : 전부 소등

BMP180 온도 드라이버

I2C 클라이언트 + IIO 서브시스템

핵심 구현

- I2C 주소 0x77 · 버스 7 (헤더 핀 3, 5)
- probe 시 Chip ID(0x55) 확인 후 보정계수 22바이트(AC1 ~ MD) 읽기
- 온도 측정 명령(0x2E) → 원시값 UT → 데이터시트 보정식으로 실제 온도 계산
- IIO 채널(TEMP, PRESSURE)로 등록 → in_temp_input (m°C 단위)
- mutex로 측정 시퀀스 보호 (동시 읽기 방지)

데이터 흐름

fan_app

▼

fopen("in_temp_input")

IIO core → read_raw()

▼

bmp180.ko

I2C 읽기 · 보정 계산

▼

BMP180 센서 (0x77)

BMP Sensor

중요 레지스터 및 장치 주소

장치 주소 ▶ 센서 선택 / 레지스터 주소 ▶ 센서 내부 레지스터 선택

레지스터 주소	값 또는 데이터	R/W	용도
0xD0	0x55 (고정값)	R	Chip ID 확인
0xAA ~ 0xBF	11개의 보정 계수 (계수당 2 byte, 총 22 bytes)	R	보정계수 가져오기
0xF4	0x2E	W	온도 측정 명령 내리기
0xF6 / 0xF7	UT의 MSB / LSB	R	온도 측정 결과 읽기

온도 측정 시작: **0xF4에 0x2E 쓰기**

I2C 장치 주소 (Linux)

0x77 = 0b1110111 (7비트)

쓰기(R/W = 0)

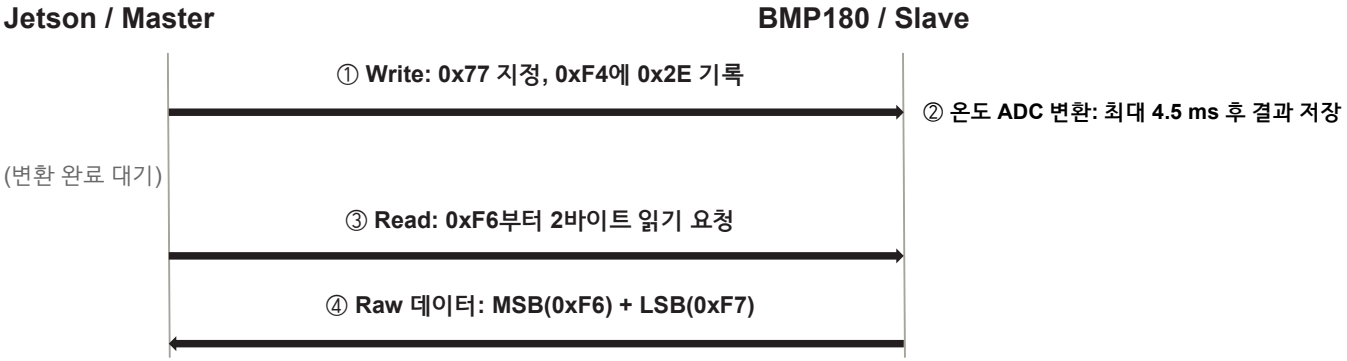
0b1110111 << 1 + 0 ▶ 0b11101110 = 0xEE

읽기(R/W = 1)

0b1110111 << 1 + 1 ▶ 0b11101111 = 0xEF

I2C 명령과 Raw 데이터 전달

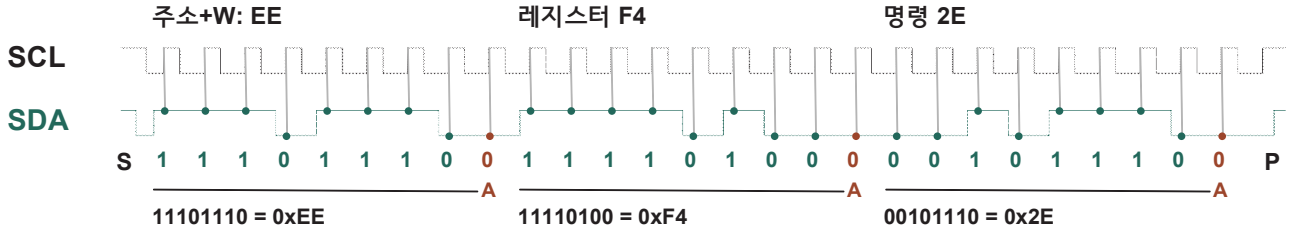
Jetson이 측정을 요청하고, 읽기 요청으로 2바이트 수신



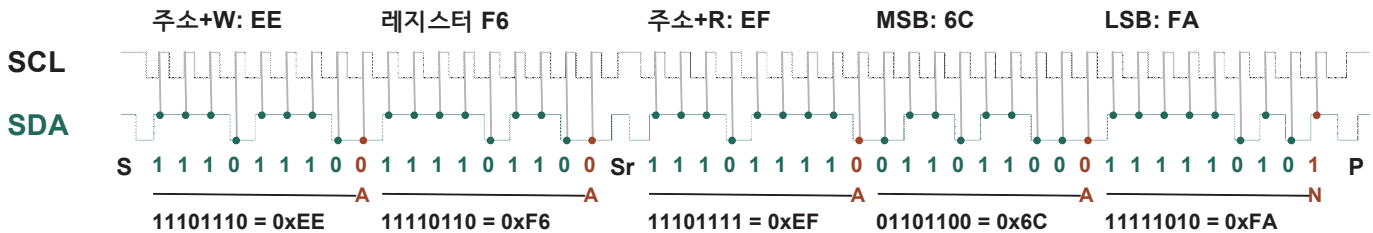
Jetson에서 UT = (MSB << 8) | LSB로 결합 후 온도 보정

SDA·SCL 파형으로 보는 I2C

① 측정 명령 쓰기



② 변환 대기 후 결과 읽기 (6C·FA는 UT 예제)

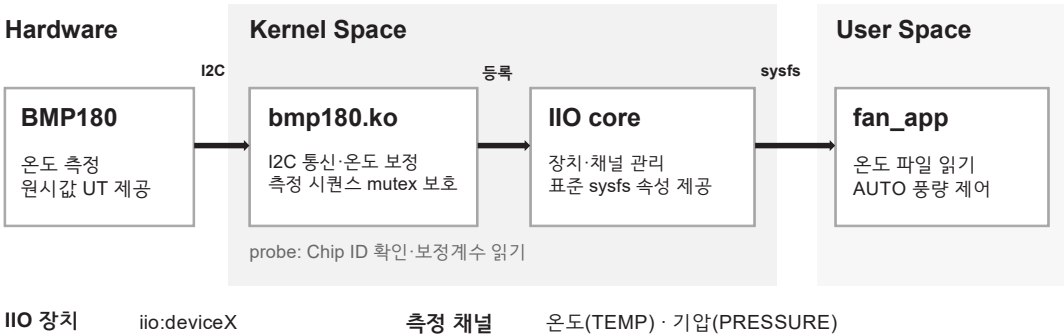


점·숫자: SCL 상승 시 SDA 값 / 밑줄: 데이터 8비트 / A·N: 별도 9번째 클록

S: START Sr: 반복 START P: STOP A: ACK(0) N: NACK(1)

IIO 구조와 각 영역의 역할

IIO (Industrial I/O): 센서 장치·채널과 앱의 측정 인터페이스를 관리하는 Linux 서브시스템

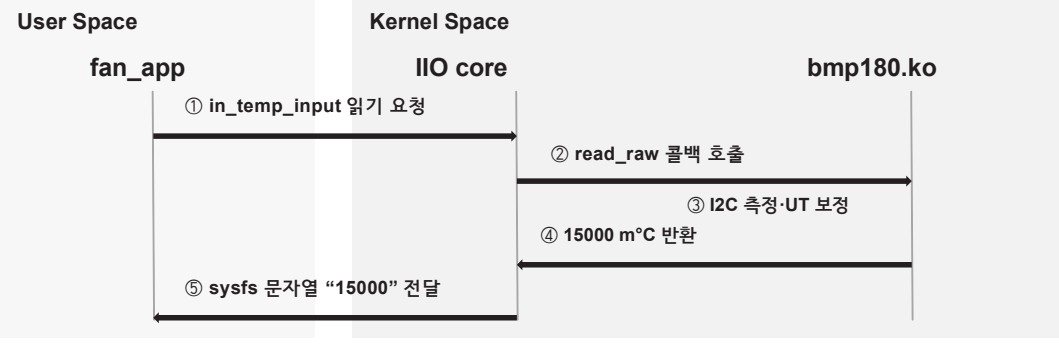


/sys/bus/iio/devices/iio:deviceX/in_temp_input

X는 등록 번호 / in_temp_input은 m°C 단위의 보정 온도 / 기압 채널 등록이 동시 측정을 뜻하지는 않음

온도 읽기부터 AUTO 풍량 제어까지

파일 읽기 요청이 IIO core를 거쳐 BMP180 드라이버로 전달되고, 보정 온도가 앱으로 돌아오는 과정



앱에서 숫자로 변환: 15000 ÷ 1000 = 15.0°C

AUTO: 1초마다 읽기 / 22°C 이상 ON · 20°C 미만 OFF / 22~32°C에서 duty 40~100%

read_raw는 콜백 이름이며, in_temp_input 요청에는 보정된 온도를 제공

Raw 데이터의 온도 보정

① 센서 레지스터 내 Raw 데이터 및 보정 계수

UT: 원시 온도 / AC5~MD: 보정계수

입력 데이터	값
UT	27898 (0x6CFA)
AC5	32757
AC6	23153
MC	−8711
MD	2868

② 입력값을 온도 보정식에 대입

$X1 = ((UT - AC6) \times AC5) / 2^{15}$

= 4743

$X2 = (MC \times 2^{11}) / (X1 + MD)$

= −2344

$B5 = X1 + X2$

= 2399

$T = (B5 + 8) / 2^{4}$

= 150

③ 보정 결과

T = 150

단위: 0.1°C → 15.0°C

④ IIO의 m°C 단위로 변환

150 × 100 = 15000 m°C

0.1°C = 100 m°C 이므로 100을 곱함

동기화 정리

드라이버별 mutex / spinlock 선택 이유

bmp180 · mutex

보호 : 측정 시퀀스 (명령 쓰기 → 변환 대기 → 읽기)

이유 : I2C 전송·대기 중 sleep → mutex

dc_motor · mutex

보호 : PWM duty · IN1/IN2 방향 설정

이유 : ioctl · release는 프로세스 문맥 → mutex

ec11 · spinlock

보호 : 회전 상태 · 키 상태 · kfifo

이유 : ISR·타이머(인터럽트 문맥)와 read() 공유

ledbar · spinlock

보호 : 표시 패턴 · 애니메이션 상태 · GPIO 출력

이유 : 타이머 콜백(softirq)과 ioctl 공유

sleep 가능한 경로(프로세스 문맥) → mutex | **인터럽트 · 타이머 문맥과 공유 → spinlock_irqsave**

메인 애플리케이션

fan_app — poll() 기반 이벤트 루프 → 4단계 반복

01 초기화

/dev/mydev · /dev/ec11
/dev/ledbar 열기
모터 정지 상태로 시작

→

02 이벤트 대기

poll(/dev/ec11, 100 ms)
입력이 없으면
타임아웃 후 다음 루프

→

03 입력 처리

회전 → duty ±5%
클릭 → 모드 전환
롱프레스 → WAIT

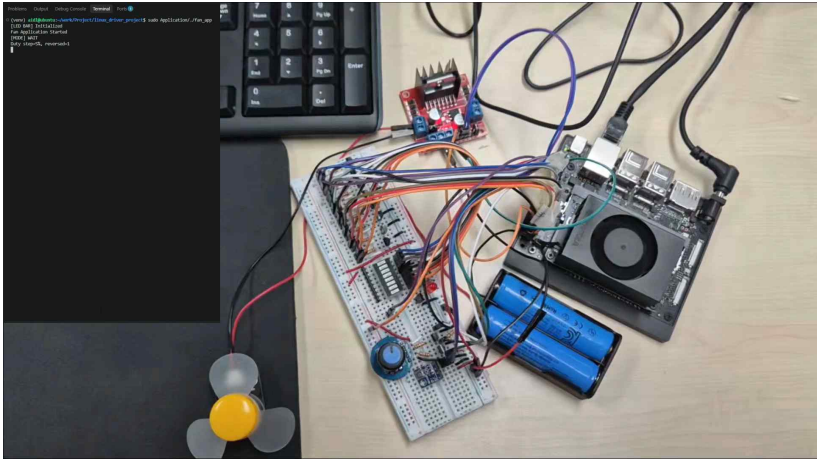
→

04 출력

ioctl → 모터 duty
ledbar_ctrl_update()
→ LED 바 갱신

AUTO 모드에서는 1초마다 BMP180 온도를 읽어 duty를 다시 계산합니다.
Ctrl+C(SIGINT) 입력 시 모터를 정지하고 LED를 끈 뒤 안전하게 종료합니다.

시연 영상



시연 순서

1. WAIT에서 시작 — 모터 정지, LED 꺼짐
2. 클릭 → MANUAL — 노브로 풍량 조절, LED 칸 수 변화
3. 클릭 → AUTO — 센서를 손으로 데워 온도 상승 → 풍량·LED 증가
4. 2초 길게 누르기 → WAIT — 모터 정지, LED 소등
5. 클릭 시 AUTO mode로 복귀
6. 클릭 → MANUAL → 노브로 풍량 조절

트러블슈팅

개발 중 겪은 문제와 해결 방법

GPIO에 값을 써도 LED가 안 켜짐

JetPack 6에서 헤더 핀이 tristate(출력 꺼짐)로 부팅됨
→ pinconf-groups로 확인 후 jetson-io로 GPIO 설정

DT 오버레이가 적용되지 않음

보드 compatible이 'p3767-0005-super'로 목록과 불일치
→ compatible 수정, 개발 중에는 gpiod lookup table로 우회

엔코더 S1·S2 신호가 읽히지 않음 (10kΩ 풀업)

풀업이 약해 NPN 반전단의 HIGH 레벨이 확보되지 않음
→ 풀업을 330Ω으로 교체해 정상 동작 확인

온도 경계에서 LED 바 깜빡임

칸 경계 온도에서 칸 수가 오르내림
→ ±0.2°C 히스테리시스로 떨림 제거

결과 및 발전 방향

구현 결과와 앞으로 개선할 점

구현 결과

- 커널 드라이버 4종(BMP180 · DC 모터 · EC11 · LED 바) 구현
- WAIT · AUTO · MANUAL 3가지 모드 통합 동작 확인
- 앱은 표준 인터페이스(IIO · ioctl · poll)로만 하드웨어 접근
- 드라이버별 단독 테스트 앱으로 분리 검증

발전 방향

- GPIO 번호 하드코딩 → 디바이스 트리 기반으로 전환
- 모드 LED를 메인 앱에 통합
- 꺼짐 예약 · 자연풍 모드 추가
- 습도 센서를 더해 불쾌지수 기반 제어

직접 작성한 리눅스 드라이버로 '센서 → 판단 → 구동'이 이어지는 임베디드 시스템을 완성했습니다



발표를 마치겠습니다

Thank You
감사합니다!